Auditable Failure Memory for ROS 2 Navigation: An Exploratory Comparison with Spatial Caching

Anonymous Authors

Abstract

Problem: In multi-phase indoor robot navigation, standard reactive navigation lacking cross-phase memory (represented by baseline R in our benchmark) repeatedly re-enters occluded blocked corridors across sequential tasks when initial line-of-sight is obstructed.
Implementation: We present **FailMem**, an event-driven failure memory architecture for ROS 2 Nav2. FailMem binds navigation aborts to immutable observation bundles (raw LiDAR scans, TF transforms, and 2D costmap regions of interest) and introduces perception-driven dynamic invalidation that unsuppresses routes upon sensor-verified clearance. Memory retention and invalidation are evaluated across sequential phases within a single continuous runner execution session.
Experiment: In physical Gazebo 11 simulations with a TurtleBot3 robot, we evaluate FailMem across 30 runs ($n = 3$ per condition under sequential fixed-seed execution) against reactive (R), spatial observation caching (O), and persistent suppression ($M1$) baselines in an asymmetric dual-path environment, alongside an exploratory 4-run feasibility check on action-conditioned navigation (H_1).
Key Results: In obstructed environments (Scenario D1), historical information eliminates redundant dead-end exploration (0.0 vs. 1.0 traversals), saving -18.5% travel distance ($14.03\,\mathrm{m}$ vs. $17.22\,\mathrm{m}$) and -22.3% simulation duration ($109.5\,\mathrm{s}$ vs. $141.0\,\mathrm{s}$) relative to reactive retry (R). Upon blockage removal (Scenario D2), dynamic invalidation reduces distance by $\approx 11.0\%$ relative to permanent suppression ($M1$) ($16.25\,\mathrm{m}$ vs. $18.25\,\mathrm{m}$), while average total simulation duration increases slightly by $\approx 1.5\%$ ($151.1\,\mathrm{s}$ vs. $148.8\,\mathrm{s}$), where fine-grained odometry speed profiling reveals that a 9.7% moving-time reduction was offset by additional chokepoint negotiation and multi-stage waypoint settling. Under static 2D geometry, FailMem and spatial caching (O) chose identical topological routes with $< 1\%$ differences in reported distance and duration ($14.03\,\mathrm{m}$ vs. $13.95\,\mathrm{m}$ in D1; $16.25\,\mathrm{m}$ vs. $16.40\,\mathrm{m}$ in D2).
Limitations: In this layout and current run sample, no additional benefit of FailMem over spatial caching (O) was observed; this does not imply general lack of benefit across arbitrary domains or statistical equivalence. Furthermore, candidate action configurations in H_1 did not establish executability divergence (4/4 Nav2 success).

1. Introduction

Autonomous mobile robots operating in structured indoor facilities—such as warehouses, hospitals, and office corridors—must repeatedly navigate shared corridors and doorways. In dynamic environments, unmapped physical blockages (such as parked delivery carts or closed safety doors) frequently obstruct the nominal shortest path. Standard navigation systems, including the ROS 2 Navigation Stack (Nav2) (Steve Macenski et al. 2020; Steven Macenski et al. 2022), utilize layered 2D costmaps (Lu, Hershberger, and Smart 2014) and local recovery behaviors (e.g., in-place rotations, backup maneuvers, costmap clearing). While these recovery routines assist during active trajectory tracking within an individual goal execution, their diagnostic state is discarded upon goal termination. Consequently, in multi-phase tasks where goals are dispatched sequentially from a distant decision junction, a purely reactive agent (lacking cross-phase memory) repeatedly re-enters known dead ends before on-board sensors re-acquire line-of-sight.

To mitigate unguided re-exploration, two primary approaches exist: 1. **Spatial Costmap Caching (O)**: Maintaining geometric occupancy in persistent global grids (e.g., Costmap2D (Lu, Hershberger, and Smart

2014), OctoMap (Hornung et al. 2013), Voxblox (Oleynikova et al. 2017)) as sensory observations accumulate. 2. **Episodic Failure Memory (F)**: Recording explicit execution aborts, error causes, or symbolic constraints to suppress infeasible route options in high-level dispatching.

While embodied AI frameworks explore language-based reflection (e.g., Reflexion (Shinn et al. 2023), REFLECT (Liu, Bahety, and Song 2023)), they operate primarily in semantic prompt spaces and lack physical costmap expiration semantics. Conversely, robotics systems rarely formalize verifiable contracts for when episodic failure records should be dynamically invalidated by physical perception. This raises an essential empirical question: *Under what conditions does an autonomous mobile robot genuinely require explicit episodic failure memory over standard spatial costmap caching?*

In this work, we present **FailMem**, an event-driven failure memory architecture for ROS 2 Nav2, and conduct an exploratory empirical study to delineate its operational boundaries. Specifically, we provide: 1. **Event-Driven Failure Memory Architecture**: A ROS 2 Nav2 action-client implementation that binds execution aborts to immutable observation bundles (raw LiDAR scans, TF transforms, costmap ROIs) and provides sensor-driven dynamic invalidation upon verified clearance. 2. **Reproducible Evaluation Benchmark**: An exploratory 30-run benchmark ($n = 3$ per condition under sequential fixed seeds) in Gazebo 11 comparing FailMem (F) against reactive (R), spatial caching (O), and persistent suppression ($M1$) baselines across clean (D0), blocked (D1), and restored (D2) scenarios. 3. **Empirical Boundary Analysis**: We demonstrate that while historical information eliminates dead ends (-18.5% distance vs. R) and dynamic invalidation reduces distance by $\approx 11.0\%$ relative to permanent suppression ($M1$) upon clearance (though average simulation duration increased by $\approx 1.5\%$), FailMem and spatial caching (O) chose identical topological routes with $< 1\%$ differences in distance and duration in static 2D geometry; in this layout and current run sample, no additional benefit of F over O was observed. Furthermore, an exploratory 4-run feasibility trial on action-conditioned navigation (H_1) showed that candidate configurations did not establish executability divergence (4/4 Nav2 success), establishing empirical stopping criteria for this exploratory branch.

---

2. Related Work & Mechanism Taxonomy

2.1 Spatial Occupancy Mapping & Distance Fields

Layered 2D costmaps (Lu, Hershberger, and Smart 2014; Steve Macenski et al. 2020) maintain geometric obstacles by ray-casting sensor ranges into grid cells and propagating configuration-space inflation. 3D volumetric frameworks, such as OctoMap (Hornung et al. 2013), employ hierarchical octrees with discrete probabilistic voxel occupancy updates. Distance-field frameworks such as Voxblox (Oleynikova et al. 2017) incrementally construct continuous Euclidean Signed Distance Fields (ESDF) for smooth trajectory optimization. While spatial mapping accurately maintains geometric free and occupied space, it treats unobserved areas identically and does not capture action-specific execution failures.

2.2 Execution Monitoring & Plan Diagnosis

Deliberative robotics frameworks (Fikes and Nilsson 1971; Simmons et al. 1998; Ingrand and Ghallab 2017) formalize plan execution monitoring and diagnostic repair. Modern navigation stacks (Steve Macenski et al. 2020) execute recovery behaviors when local planners fail. However, standard recovery states are discarded upon goal termination, requiring explicit cross-phase or cross-task episodic memory stores to guide high-level routing.

2.3 LLM Self-Reflection & Agent Memory

Recent embodied AI systems investigate large language models for planning and error correction. Reflexion (Shinn et al. 2023) prompts LLMs with scalar task feedback to generate natural language self-reflections. Inner Monologue (Huang et al. 2023) integrates success detectors and scene descriptions into closed-loop

prompt chains. REFLECT (Liu, Bahety, and Song 2023) uses hierarchical multi-modal experience summarization with LLMs to generate structured failure explanations and plan corrections. While these methods demonstrate prompt-level flexibility, they focus on higher-level task reflection. FailMem complements these approaches by investigating event-driven failure recording, perception-driven invalidation, and offline replay auditing directly within the low-level ROS 2 action and costmap stack.

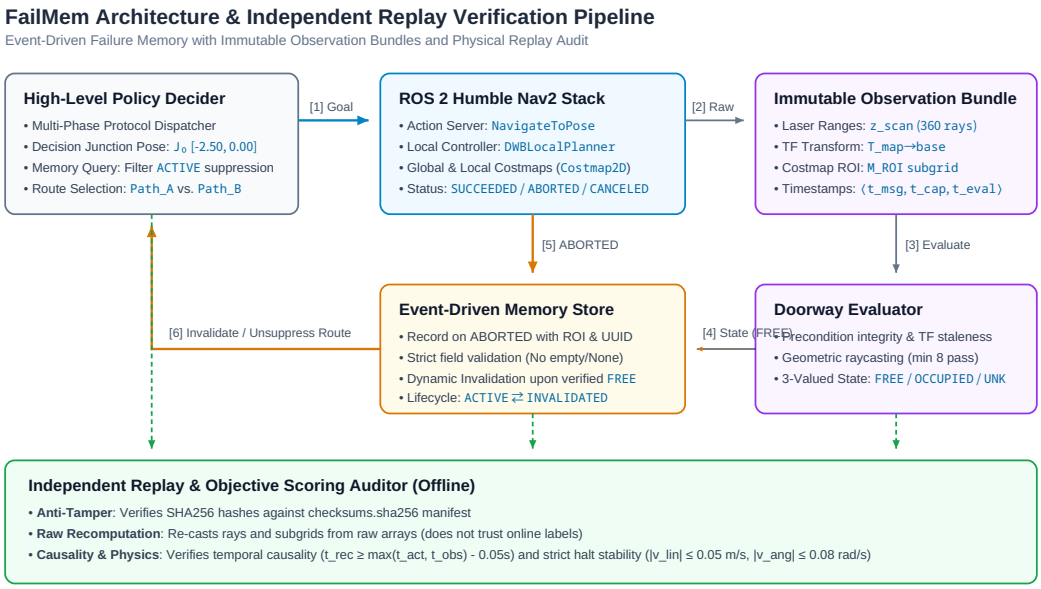
2.4 Experience-Based Planning & No-Good Learning

In motion planning, the Lightning framework (Berenson, Srinivasa, and Kuffner 2012) and Experience Graphs (E-Graphs) (Phillips et al. 2012) reuse libraries of prior successful paths to accelerate search. In discrete optimization, conflict-driven clause learning (e.g., GRASP (Marques-Silva and Sakallah 1999)) records “no-goods” from search dead ends to prune infeasible branches. FailMem adapts the principle of negative constraint recording to mobile robot navigation with explicit sensor-verified invalidation.

Table 1 summarizes the key characteristics across representations and invalidation mechanisms.

Paradigm	Representation	Grounding Level	Invalidation Trigger
Costmap2D / Nav2	2D Occupancy Grid	Geometric cells	Direct laser ray traversal
OctoMap	3D Voxel Octree	Discrete 3D voxels	Direct laser ray traversal
Voxblox	Continuous ESDF / TSDF	3D Distance Field	Dynamic voxel integration
Experience Graphs	Path Waypoints / Graph	Discrete Path	Collision check on repair
Reflexion	Natural Language Text	Semantic tokens	Manual context reset
REFLECT	Natural Language Text	Multi-modal logs	Session termination
FailMem (Implemented)	Event Tuple	Raw sensor bundle	Sensor-verified `FREE` state
FailMem F2 (Proposed)	Profile Tuple	Bundle + Profile	Profile capability check

3. The FailMem Architecture



FailMem Architecture and Verification Pipeline

Figure 1 illustrates the overall system architecture, encompassing the execution runtime, immutable observation bundles, and memory lifecycle.

3.1 Immutable Observation Bundles

Prior to evaluating passage clearance, FailMem constructs an immutable observation bundle B:

$$B = \langle \text{obs_id}, \mathbf{z}_{\text{scan}}, \mathbf{T}_{\text{map}\to\text{base}}, \mathbf{M}_{\text{costmap_ROI}}, t_{\text{msg}}, t_{\text{capture}}, t_{\text{eval}} \rangle$$

where $\mathbf{z}_{\text{scan}}$ is the raw range array, $\mathbf{T}_{\text{map}\to\text{base}}$ is the TF transformation, and $\mathbf{M}_{\text{costmap_ROI}}$ is the extracted 2D subgrid matrix.

3.2 Clearance Classification Rules

The passage state is evaluated according to strict geometric criteria implemented in `src/doorway_evaluator.py`:

$$\text{State}(B) = \begin{cases} \text{OCCUPIED}, & \text{if } N_{\text{hits_inside}} \geq 5 \text{ or } N_{\text{costmap_occupied_cells}} > 0 \\ \text{FREE}, & \text{if } N_{\text{hits_inside}} = 0 \text{ and } N_{\text{pass_through}} \geq 8 \text{ and } \text{CostmapCleared}(B) \\ \text{UNKNOWN}, & \text{otherwise} \end{cases}$$

where $\text{CostmapCleared}(B) \iff (N_{\text{unknown_cells}} = 0 \land N_{\text{occupied_cells}} = 0)$ within the doorway ROI $[-0.15, 0.15, 0.95, 1.45]$. (See Supplementary Material S1 for detailed transform alignment and ray projection preconditions).

3.3 Memory Lifecycle & Invalidation Mechanics

- **Failure Registration**: When navigation fails within an occupied passage, an active failure record is created:

  $$M = \langle \text{mem_id}, t_{\text{fail}}, \text{goal_uuid}, \text{action_id}, \text{region_id}, \mathbf{M}_{\text{costmap_ROI}} \rangle$$

  Active failure records suppress the associated route at Decision Junction J_0.
- **Dynamic Invalidation**: When subsequent sensor perception yields $\text{State}(B) = \text{FREE}$ with evidence timestamp $t_{\text{free}} > t_{\text{fail}}$, the memory entry is marked INVALIDATED, unsuppressing the route.
- **Cross-Phase Scope Clarification**: In the current experimental implementation, history collection, memory creation, decision dispatch, and memory reset occur within a single continuous runner session across sequential phases. This evaluates phase-to-phase memory transfer within an experimental run, and has not yet been benchmarked as a persistent multi-task database across separate OS daemon lifetimes.
- **Evaluation Pipeline**: An independent cryptographic replay auditor verifies all serialized logs against SHA256 checksum manifests and physical halt stability criteria ($|v_{\text{lin}}| \leq 0.05\,\text{m/s}, |v_{\text{ang}}| \leq 0.08\,\text{rad/s}$) without reliance on runtime assertions (detailed in Supplementary Material S2).

---

4. Experimental Setup & Benchmarking Protocol

4.1 Physical Simulation Environment & Geometry

Simulations are conducted in ROS 2 Humble with Gazebo 11 simulating a TurtleBot3 Waffle robot in an asymmetric dual-path environment (`configs/p2c_dualpath_world.model`). - **Path A (North Short Path)**: Waypoint polyline length is $6.124\,\text{m}$ from Decision Junction $J_0[-2.50, 0.00]$ to Goal $[2.50, 0.00]$ through a $0.80\,\text{m}$ doorway chokepoint at $y = 1.20$. - **Path B (South Bypass Detour)**: Waypoint polyline length is $9.105\,\text{m}$ from J_0 to Goal via open corridor at $y = -2.40$.

4.2 Distance Metric Definitions

To evaluate performance across different phases, we define: 1. **Decision Distance (d_{dec})**: Physical odometry distance traversed during Phase 2 after goal dispatch from Decision Junction J_0 (Path B direct: $\approx 7.54\,\text{m}$; Path A direct: $\approx 5.89\,\text{m}$). 2. **Total Distance (d_{tot})**: Cumulative odometry distance spanning both Phase 1 (historical observation/attempt) and Phase 2 (dispatch to goal).

4.3 Evaluated Scenarios & Baseline Methods

- **Scenario D0 (Unobstructed)**: Doorway is clear; robot navigates Path A without prior history.
- **Scenario D1 (Blocked)**: Doorway chokepoint is obstructed by a physical obstacle box. Phase 1 executes the following sequence:
  1. *Vantage Navigation*: The robot navigates from Decision Junction $J_0[-2.50, 0.00]$ to approach vantage waypoint $[-1.00, 1.20, 0.0]$ and captures initial sensor observation bundle;
  2. *Chokepoint Traversal Attempt*: The robot dispatches a genuine Nav2 action attempting chokepoint traversal to $[0.50, 1.20, 0.0]$ with a $15.0\,\text{s}$ timeout;
  3. *Action Termination*: Nav2 execution aborts due to physical obstacle obstruction;
  4. *Failure Memory Creation*: Upon action termination, the robot captures a fresh post-traversal observation bundle confirming `OCCUPIED`, classifies the failure cause, and commits an active failure memory record (or updates spatial cache O);
  5. *Structured Return*: The robot executes a return navigation action back to Decision Junction J_0.
     Phase 2 then evaluates routing dispatched from J_0. Under Reactive R, having no cross-phase memory, the robot dispatches the nominal shortest path (Path A), physically travels to entrance waypoint $[-1.50, 1.20]$, detects the blockage on live sensors, and retreats to J_0 before taking Path B. This decision-phase unguided probe in R is strictly distinct from the historical Phase 1 traversal attempt executed in F and O.
- **Scenario D2 (Cleared)**: Following D1, the obstacle is removed. Phase 1 observes `FREE` doorway clearance from vantage waypoint $[-1.00, 1.20]$ and retreats to J_0. Phase 2 evaluates routing from J_0.

Evaluated Methods: 1. **Reactive Baseline (R)**: No cross-phase memory. In D1, enters Path A approach leg, detects blockage at $[-1.50, 1.20]$, and retreats to J_0 before taking Path B. 2. **Spatial Observation Cache (O)**: Caches 2D geometric occupancy observed from vantage point during Phase 1. Evaluated at J_0 during Phase 2. 3. **FailMem (F)**: Event-driven failure memory with dynamic perception-driven invalidation. 4. **Persistent Suppression ($M1$)**: Static failure memory permanently suppressing Path A without dynamic invalidation.

All runs use $n = 3$ per condition under sequential fixed seeds with an episode simulation time budget of $180.0\,\text{s}$.

---

5. Empirical Results & Boundary Analysis

5.1 Condition-Level Performance

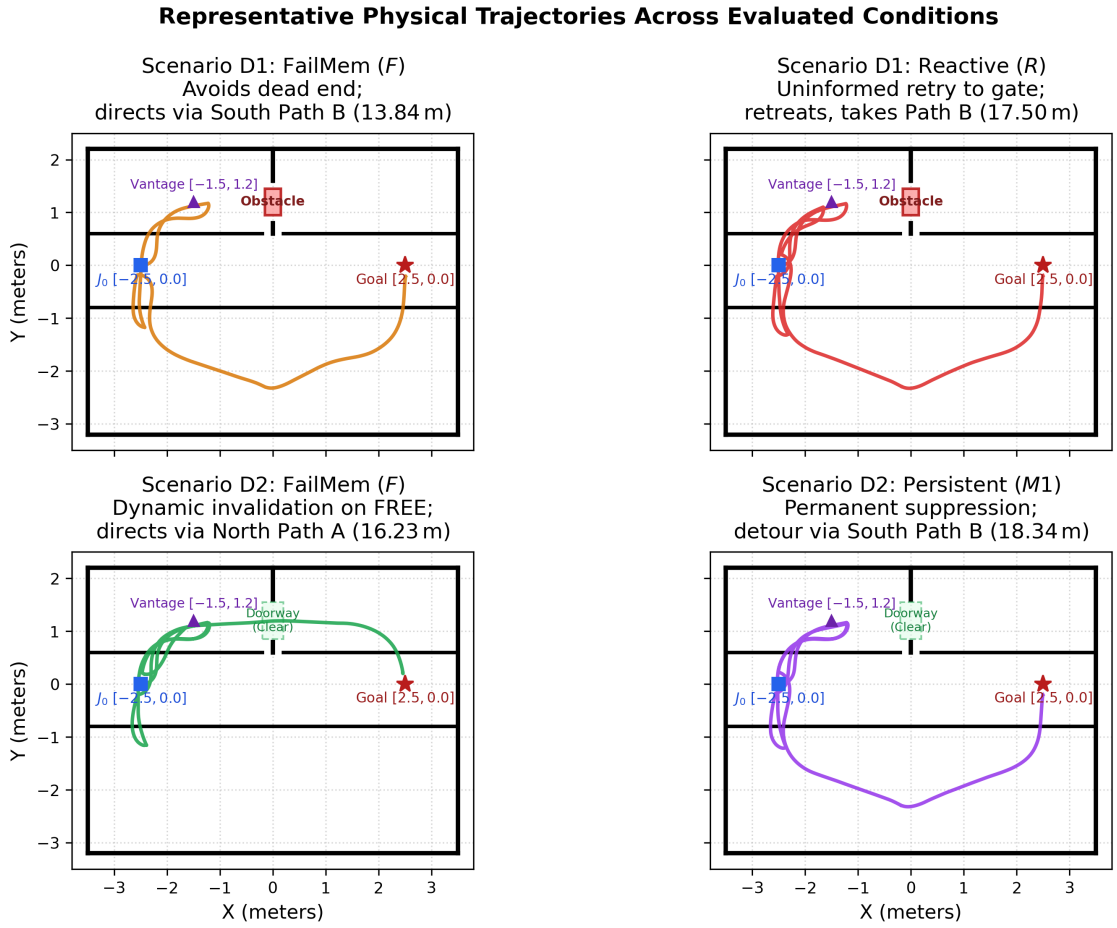
Table 2 presents condition-level performance across 30 physical simulation runs ($n = 3$ per condition, reporting mean $\pm$ sample standard deviation with Bessel correction $ddof = 1$). All 10 conditions achieved a 100% replay audit pass rate ($3/3$), confirming physical halt stability and data completeness across all runs.

Scenario	Condition	n	Actual Route	Dead-End Traversals	Decision Dist (m)	Decision Time (s)	Total Dist (m)	Total Time (s)	Replay Audit Pass
D0	R	3	`Path_A`	0.0 ± 0.0	6.160 ± 0.037	51.533 ± 1.656	6.160 ± 0.037	51.533 ± 1.656	3/3 (100%)
D0	O	3	`Path_A`	0.0 ± 0.0	6.203 ± 0.007	49.967 ± 0.850	6.203 ± 0.007	49.967 ± 0.850	3/3 (100%)
D0	F	3	`Path_A`	0.0 ± 0.0	6.196 ± 0.031	49.900 ± 0.458	6.196 ± 0.031	49.900 ± 0.458	3/3 (100%)
D1	R	3	`Path_A_then_Path_B`	1.0 ± 0.0	10.816 ± 0.072	82.067 ± 4.394	17.220 ± 0.242	141.000 ± 5.910	3/3 (100%)
D1	O	3	`Path_B`	0.0 ± 0.0	7.550 ± 0.052	50.033 ± 1.986	13.947 ± 0.070	108.700 ± 2.718	3/3 (100%)
D1	F	3	`Path_B`	0.0 ± 0.0	7.536 ± 0.003	48.200 ± 0.954	14.027 ± 0.252	109.500 ± 1.572	3/3 (100%)
D2	R	3	`Path_A`	0.0 ± 0.0	5.915 ± 0.044	49.567 ± 1.595	16.344 ± 0.032	148.167 ± 3.164	3/3 (100%)
D2	O	3	`Path_A`	0.0 ± 0.0	5.939 ± 0.066	52.033 ± 1.747	16.396 ± 0.240	149.767 ± 0.551	3/3 (100%)
D2	F	3	`Path_A`	0.0 ± 0.0	5.886 ± 0.022	51.100 ± 0.954	16.247 ± 0.098	151.100 ± 7.762	3/3 (100%)
D2	$M1$	3	`Path_B`	0.0 ± 0.0	7.895 ± 0.023	51.300 ± 1.997	18.248 ± 0.094	148.833 ± 2.397	3/3 (100%)

5.2 Pairwise Contrasts & Trajectory Analysis

Table 3 reports pairwise contrast comparisons across evaluated methods, and Figure 2 illustrates representative physical trajectories.

Scenario	Comparison	Metric	Method A Mean	Method B Mean	Abs Diff ($A - B$)	Rel Diff (%)
D1	F vs. R	Total Dist (m)	14.027	17.220	-3.193	**-18.5%**
D1	F vs. R	Total Time (s)	109.500	141.000	-31.500	**-22.3%**
D1	O vs. R	Total Dist (m)	13.947	17.220	-3.273	**-19.0%**
D1	O vs. R	Total Time (s)	108.700	141.000	-32.300	**-22.9%**
D1	F vs. O	Total Dist (m)	14.027	13.947	+0.080	+0.6%
D1	F vs. O	Total Time (s)	109.500	108.700	+0.800	+0.7%
D2	F vs. $M1$	Total Dist (m)	16.247	18.248	-2.000	**-11.0%**
D2	F vs. $M1$	Total Time (s)	151.100	148.833	+2.267	+1.5%
D2	F vs. O	Total Dist (m)	16.247	16.396	-0.149	-0.9%
D2	F vs. O	Total Time (s)	151.100	149.767	+1.333	+0.9%



Representative Physical Trajectories Across Evaluated Conditions

1. **Elimination of Redundant Dead-End Exploration (F vs. R)**: In Scenario D1, FailMem (F) eliminates redundant entry into the blocked corridor (0.0 vs. 1.0 dead ends), reducing total distance by -18.5% ($14.03\,\mathrm{m}$ vs. $17.22\,\mathrm{m}$) and simulation duration by -22.3% ($109.5\,\mathrm{s}$ vs. $141.0\,\mathrm{s}$) relative to Reactive R (Table 3, Figure 2). Reactive R's dead-end traversal represents an unguided retry to the entrance waypoint, not physical collision.
2. **Prevention of Detour Overhead (F vs. $M1$)**: In Scenario D2, dynamic invalidation upon observing `FREE` unsuppresses Path A, reducing total distance by $\approx 11.0\%$ ($2.00\,\mathrm{m}$, $16.25\,\mathrm{m}$ vs. $18.25\,\mathrm{m}$) relative to persistent suppression ($M1$). However, average total simulation duration increased slightly by $\approx 1.5\%$ ($151.1\,\mathrm{s}$ vs. $148.8\,\mathrm{s}$). Fine-grained odometry speed profile decomposition reveals the physical mechanism: FailMem on Path A reduced active moving time from $75.14 \pm 0.42\,\mathrm{s}$ to $67.85 \pm 0.92\,\mathrm{s}$ ($-7.29\,\mathrm{s}$, a 9.7% reduction, matching the shorter trajectory). However, negotiating Path A's narrow $0.80\,\mathrm{m}$ doorway chokepoint required 3 multi-stage waypoints and settling stability windows ($|v_{\mathrm{lin}}| \leq 0.05\,\mathrm{m/s}$), increasing stopped/settling time from $55.70 \pm 2.81\,\mathrm{s}$ to $64.76 \pm 6.20\,\mathrm{s}$ ($+9.06\,\mathrm{s}$), whereas $M1$ traversed the wide, open bypass corridor (Path B) with only 2 continuous actions. Turning durations remained virtually identical ($17.03 \pm 0.71\,\mathrm{s}$ vs. $16.49 \pm 0.07\,\mathrm{s}$). Thus, chokepoint negotiation and multi-stage waypoint settling fully offset the active moving-time savings.
3. **Comparison with Spatial Caching in Static 2D Geometry (F vs. O)**: In both D1 and D2, FailMem (F) and Spatial Cache (O) selected identical topological routes. Total distance differences ($+0.6\%$ in D1, -0.9% in D2) and total duration differences ($+0.7\%$ in D1, $+0.9\%$ in D2) remained under 1%. *In this layout and current run sample, no additional benefit of F over O was observed; this does not imply*

general lack of benefit across arbitrary domains or statistical equivalence. Notably, dynamic recovery upon obstacle removal is not exclusive to FailMem; spatial costmap caching (O) likewise updates its internal occupancy state and unblocks routes upon acquiring new sensory observations.

5.3 Feasibility Check for Action-Conditioned Navigation (H_1)

To investigate whether action configurations (aligned a_{aligned} vs. doorpost boundary-biased oblique a_{oblique}) create executability differences in an unobstructed doorway, a fixed 4-run feasibility check was conducted (`reports/evidence/p2d_h1_feasibility/`).

Run Name	Action Profile	Target Goal	Doorway Perception	Nav2 Status (Code)	Est. Nav2 Nav Time (s)	Assumed Settling (s)	Stability Window (s)	Total Sim Duration (s)	Physical Arrival Verified
`H1_aligned_run1`	`act_aligned`	`[1.50, 1.20, 0.0]`	`FREE` (23 rays)	`SUCCEEDED` (4)	13.4 (est.)	3.0 (nom.)	2.4	18.8	**False** (excess av: 0.1068 > 0.08)
`H1_aligned_run2`	`act_aligned`	`[1.50, 1.20, 0.0]`	`FREE` (22 rays)	`SUCCEEDED` (4)	13.4 (est.)	3.0 (nom.)	2.4	18.8	**True**
`H1_oblique_run1`	`act_oblique`	`[0.50, 0.88, 0.0]`	`FREE` (23 rays)	`SUCCEEDED` (4)	11.1 (est.)	3.0 (nom.)	2.3	16.4	**True**
`H1_oblique_run2`	`act_oblique`	`[0.50, 0.88, 0.0]`	`FREE` (22 rays)	`SUCCEEDED` (4)	11.0 (est.)	3.0 (nom.)	2.3	16.3	**True**

Table 4 summarizes the feasibility trial outcomes. Nav2 returned `SUCCEEDED` (status code 4) in all 4/4 runs, and strict physical arrival was verified in 3/4 runs (`H1_aligned_run1` exceeded the angular velocity halt threshold during the stability window: $0.1068 > 0.08\,\text{rad/s}$). Nav2's `DWBLocalPlanner` navigated the passage without planner abortion across all runs.

Decision & Conclusion: *The candidate scenario did not establish the expected executability divergence across actions; exploration for this hypothesis was therefore stopped per protocol, which does not constitute a refutation of the general action-conditioned failure memory hypothesis.*

---

6. Threats to Validity & Limitations

1. **Exploratory Sample Size**: The 30-run comparative pilot ($n = 3$ per condition) serves as an exploratory demonstration of mechanism functioning and auditability; it does not constitute large-sample asymptotic hypothesis testing.
2. **Simulation Fidelity**: Evaluated in Gazebo 11 simulation with nominal sensor models. Hardware deployments with real-world sensor dropout, dynamic obstacles, and adversarial conditions are subject to future research.
3. **Conceptual Extensions ($F2, O+$)**: Action-profiled failure memory ($F2$) and backoff spatial caching ($O+$) are formalized as conceptual models in project documentation but have not yet been evaluated in simulation.

4. **Within-Session Phase-Level Scope**: Memory evaluation is restricted to sequential phases within a single continuous experimental execution session. Cross-task persistence across operating system restarts, long-term storage indexing, and multi-day operation remain unverified.

7. Conclusion

This paper presented FailMem, an event-driven failure memory architecture for autonomous mobile robots. By binding execution failures to immutable observation bundles and providing an independent replay audit pipeline, FailMem establishes rigorous standards for reproducible robotics evaluation. Our empirical findings demonstrate that historical information eliminates redundant dead-end exploration relative to reactive retry, and dynamic state updates (available in both FailMem and spatial caching) prevent permanent detour traps upon environmental recovery. In the evaluated layout and run sample, no additional routing benefit of explicit failure memory over spatial caching was observed. These findings provide an auditable, reproducible benchmark and empirical stopping rationale to guide future research on action-conditioned and multi-agent memory systems.

References

Berenson, Dmitry, Siddhartha S. Srinivasa, and James Kuffner. 2012. “A Robot Path Planning Framework for Adaptive Experience Reuse.” *The International Journal of Robotics Research* 31 (10): 1237–54. https://doi.org/10.1177/0278364912456311.

Fikes, Richard E., and Nils J. Nilsson. 1971. “STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving.” *Artificial Intelligence* 2 (3-4): 189–208. https://doi.org/10.1016/0004-3702(71)90010-5.

Hornung, Armin, Kai M. Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. 2013. “OctoMap: An Efficient Probabilistic 3d Mapping Framework Based on Octrees.” *Autonomous Robots* 34 (3): 189–206. https://doi.org/10.1007/s10514-012-9321-0.

Huang, Wenlong, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, et al. 2023. “Inner Monologue: Embodied Reasoning Through Planning with Language Models.” In *Conference on Robot Learning (CoRL)*, 205:1769–82. Proceedings of Machine Learning Research. PMLR.

Ingrand, Félix, and Malik Ghallab. 2017. “Deliberation for Autonomous Robots: A Survey.” *Artificial Intelligence* 247: 10–44. https://doi.org/10.1016/j.artint.2014.11.003.

Liu, Zeyi, Arpit Bahety, and Shuran Song. 2023. “REFLECT: Summarizing Robot Experiences for Failure Explanation and Correction.” In *Conference on Robot Learning (CoRL)*, 229:3468–84. Proceedings of Machine Learning Research. PMLR.

Lu, David V., Dave Hershberger, and William D. Smart. 2014. “Layered Costmaps for Context-Sensitive Navigation.” In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 709–15. IEEE. https://doi.org/10.1109/IROS.2014.6942636.

Macenski, Steve, Francisco Martín, Ruediger White, and Jonatan Ginés Clavero. 2020. “The Marathon 2: A Navigation System.” In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2718–25. IEEE. https://doi.org/10.1109/IROS45743.2020.9341207.

Macenski, Steven, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. 2022. “Robot Operating System 2: Design, Architecture, and Uses in the Wild.” *Science Robotics* 7 (66): eabm6074. https://doi.org/10.1126/scirobotics.abm6074.

Marques-Silva, João P., and Karem A. Sakallah. 1999. “GRASP: A Search Algorithm for Propositional Satisfiability.” *IEEE Transactions on Computers* 48 (5): 506–21. https://doi.org/10.1109/12.769433.

Oleynikova, Helen, Zachary Taylor, Marius Fehr, Roland Siegwart, and Juan Nieto. 2017. “Voxblox: Incremental 3d Euclidean Signed Distance Fields for on-Robot Exploration.” In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1366–73. IEEE. https://doi.org/10.1109/IROS.2017.8202315.

Phillips, Mike, Benjamin J. Cohen, Sachin Chitta, and Maxim Likhachev. 2012. “E-Graphs: Bootstrapping Planning with Experience Graphs.” *The International Journal of Robotics Research* 31 (10): 1159–75. https://doi.org/10.1177/0278364912461942.

Shinn, Noah, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. “Reflexion: Language Agents with Verbal Reinforcement Learning.” In *Advances in Neural Information Processing Systems (NeurIPS)*, 36:8634–52.

Simmons, Reid, Richard Goodwin, Karen Zita Haigh, Sven Koenig, and Joseph O’Sullivan. 1998. “The Xavier Mobile Robot: Autonomous Navigation in Human Environments.” *IEEE Robotics & Automation Magazine* 5 (1): 34–42. https://doi.org/10.1109/100.740882.